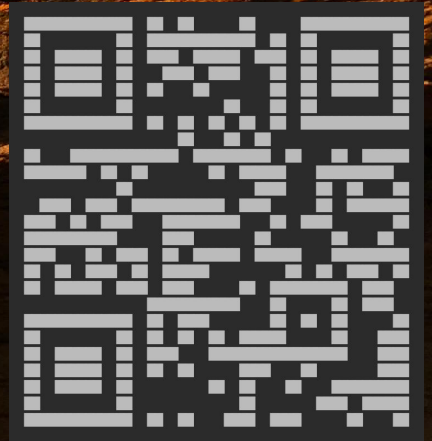


Working with the AV1 Codec

Prepared for syd<video>
Hilton Sydney October 2018

by Kevin Staunton-Lambert
@keвлеyski

pyrmontbrewery.com.au



Okay, explain video “**codec**” like I’m five...

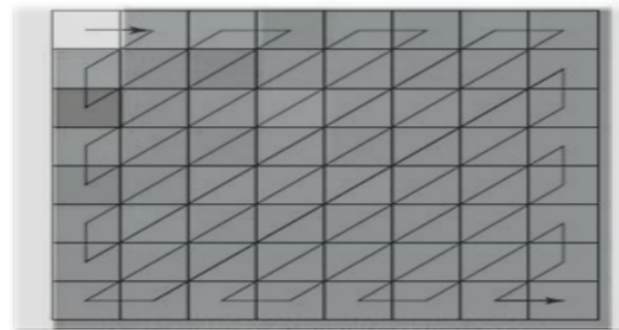


Grown-up analogy... compressing the photos

QUANTIZE COEFFICIENTS

239 32 34 -70 -3 27 -12 -19 2 ...

= Intra fame
(keyframe)



DCT COEFFICIENTS

239	-32	27	-12	3	-5	3	-1
24	-1	-19	6	2	0	-1	1
-70	2	8	23	0	6	-1	-1
5	0	-6	11	-2	0	-1	1
-17	-2	6	6	2	-1	0	0
2	-4	2	2	1	-2	0	1
-3	0	0	-1	-1	-1	0	0
1	-1	3	1	0	0	0	0

Grown-up analogy... compressing the flick book
strengths and weaknesses



So, what's all the fuss about **AV1 codec**?



But, couldn't we just *pay* for MPEG HEVC/H.265?

Well, you could

**... Everybody realises that the old
MPEG business model is now broke."**

*[Leonardo Chiariglione, the founder and
chairman of MPEG]*

H.265
HEVC
High Efficiency



No worries, what's **AOMedia**?



Jan
2018



ARGON DESIGN



libaom

libaom is effectively a set of tools - referred to 'experiments' - which you can find freely available here: <https://aomedia.googlesource.com/aom>

Bugs (Monorail): <https://bugs.chromium.org/p/aomedia/issues/list>

Builds for any platform using CMake (>= 3.5) - example: `cmake ../aom`

`-DCONFIG_DEBUG=1`

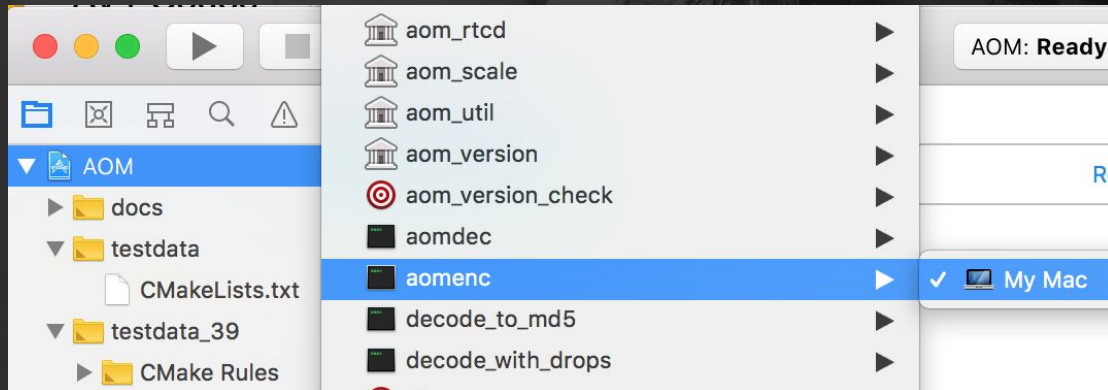
Cross compile gcc/clang/lvmm

Various IDE integrations

e.g. Xcode

```
cmake -G Xcode
```

```
open AOM.xcodeproj
```



Encoding: aomenc (libaom command line tool)

Encoder works on raw **yuv4mpeg** (e.g. `ffmpeg -i in.mp4 --pix_fmt yuv422p raw.y4m`)

```
aomenc -v -w 3840 -h 2160 --cpu-used=0 --target-bitrate=1500 -t 16 -u 0 -b 10 --fps=60 -o av1.webm raw.y4m
```

(tool output) Pass 1/2 frame 4/3 552B 11382 ms 21.08 fpm [ETA 48:16:11] or maybe a *lot* longer ;-)

Rate Control options	<code>--target-bitrate</code> (kbps)
Bitstream profile	<code>-u</code> (<code>-u 0 =main</code> , <code>-u 1 =high</code> , <code>-u 2 =professional</code>)
Bit depth	<code>-b</code> (<code>-b 8</code> , <code>-b 10</code> or <code>-b 12</code>)
Aspect, frame size	<code>-w -h</code> (e.g. 4K <code>-w 3840 -h 2160</code>)
Frame rate	<code>--fps</code> (e.g. <code>--fps=60</code>)
Keyframe Placement	<code>--kf-min-dist</code> (e.g. <code>--kf-min-dist=360</code> key frame every 6 seconds)
Multi-threaded op	<code>-t</code> (<code>-t 16</code> use up to 16 threads) <code>--cpu-used=0</code>
+ heaps of other options (see <code>aomenc --help</code>)	

Encoding: AV1 Bitstream **profiles** and **levels**

Profiles and levels specify restrictions on the capabilities needed to decode.

“**main**” - supports YUV 4:2:0 or monochrome streams with bit-depths 8 or 10.

“**high**” - adds support for YUV 4:4:4 streams with the same bit-depth

“**professional**” - bit-depth 12 + support for the YUV 4:2:2 video format.

Various levels, here are some examples...

1080p	Level 4.0	32 tiles	< 12Mbps (main)	< 30Mbps (high)	1920x1080@30fps
4K	Level 5.0	64	< 30Mbps	< 100Mbps	3840x2160@30fps
8K	Level 6.0	128	< 60Mbps	< 240Mbps	7680x4320@30fps

Encoding: AV1 Digital Rights Management

AV1 is supported by **ISO BMFF** and **WebM** (Matroska) and so implies **common encryption (CENC)** / AES-128 based encryption of the payload

Both **cenc** and **cbcs** scheme types are permitted.

All **Open Bitstream Unit (OBU)** headers are *unencrypted* which makes manipulating encrypted AV1 streams easy.

Similarly Temporal Delimiter, Sequence Header, Metadata, except for those requiring protection are also unencrypted allowing easy processing and transport of streams containing encrypted AV1 payloads.

Tile groups are individually encrypted (so encryption can be parallel/concurrent)

Encoding: **FFmpeg** 4.0 (**libaom-av1** encoder)

First, you'll need to build latest ffmpeg yourself with `--enable-aom`

```
ffmpeg -i input.mp4 -c:v libaom-av1 -strict -2 -y av1.mkv
```

note, this is still considered experimental, hence `-strict -2`

Some parameters that get passed through to libaom (note there are many that are not today passed on)

Quality/Speed ratio modifier	<code>-cpu-used</code> from -8 to 8 (default 1)
Alternate reference frames	<code>-auto-alt-ref</code> enable use of alternate frames 2-pass only, from -1 to 2 (default -1)
Number of frames to look ahead	<code>-lag-in-frames</code> at for alternate reference frame selection from -1 to INT_MAX (def -1)
Constant quality mode	<code>-crf</code> select the quality from -1 to 63 (default -1)
Change threshold to skip	<code>-static-thresh</code> when blocks should be skipped by the encoder, 0 to INT_MAX (def 0)
Frame drop threshold	<code>-drop-threshold</code> from INT_MIN to INT_MAX (default 0)
Noise sensitivity	<code>-noise-sensitivity</code> noise sensitivity (from 0 to 4) (default 0)
Error resilience configuration	<code>-error-resilience</code> (default) losses of whole frames (partitions) frame partitions

Encoding: **GStreamer** 1.14 (**av1enc** element)

Libaom is wrapped in the “aom” bad plug-in right now, it’s based on the libvpx plugin element which is why currently it sits in that category until reviewed

Again you need to build this yourself from sources

```
gst-launch-1.0 filesrc location=input.mp4 !  
video/x-raw,width=3840,height=2160 ! av1enc ! queue !  
filesink location=av1.mp4 -v -e
```

Encoding: **rav1e**

Xipf



Transport: AV1 over **MPEG-DASH**

ISOBMFF (MP4) containers already support AV1 payloads
WebM files can also play under Firefox Nightly

```
<Period id="0" start="PT0S" duration="PT175.014S" >
  <AdaptationSet id="0" mimeType="video/mp4" codecs="av1.experimental.[version]"
    width="3840" height="2160" subsegmentAlignment="true" >
    <Representation id="0" bandwidth="1500000">
      <BaseURL>av1.mp4</BaseURL>
      <SegmentBase indexRange="6978450-6978478">
        <Initialization range="0-1116" />
      </SegmentBase>
    </Representation>
  </AdaptationSet>
</Period>
```

Transport: AV1 over **HLS/FPS** (CMAF)

Whilst Apple devices *do not* yet support playback of AV1 samples, we should expect this to work in the future.

HLS specification - since v5 - includes support for **fragmented MPEG-4 (fMP4)**
AV1 **HLS** segments would consist of a *Movie Fragment Box (moof)* containing a subset of the sample table and a *Media Data Box (mdat)* containing the AV1

```
#EXTM3U
#EXT-X-TARGETDURATION:175
#EXT-X-VERSION:5
#EXT-X-MEDIA-SEQUENCE:1
#EXT-X-PLAYLIST-TYPE:VOD
#EXT-X-MAP:URI="av1.mp4",BYTERANGE="1116@0"
#EXTINF:175.014,
#EXT-X-BYTERANGE:6978478@6978450
av1.mp4
```

Transport: AV1 over **WebRTC**

AV1 similar to VPx is to be included as an **RTP payload format** (which describes how the encoded AV1 bitstream is encapsulated in RTP packing)

This means AV1 encodings can be **peer2peer shared** via WebRTC, and in the future, low bitrate/**low latency** AV1 encodings for **live*** sports and conferencing will become possible.

Media delivery would be **WebM** contained AV1/**Opus**** over secure WebSockets

** Problem right now is live software encoding is just way too slow to benefit from low latency integrations, but we should expect this to be the norm in the future with dedicated hardware live encoders*

*** WebRTC requires **Opus** audio codec, based on (Skype) Speex and (Xipf) Vorbis which is also royalty-free!*

Playback: AV1 decoders

libaom - aomdec tool will decode AV1 to raw yuv4mpeg frames, or you can include the library into your own native player.

FFmpeg - ffplay tool includes libaom and so will can play AV1 4K directly

Stream #0:0: Video: **av1 (Main)**, yuv420p(tv), 3840x2160, SAR 1:1 DAR 16:9, 29.97 fps, 29.97 tbr, 1k tbn, 1k tbc (default)

Metadata:

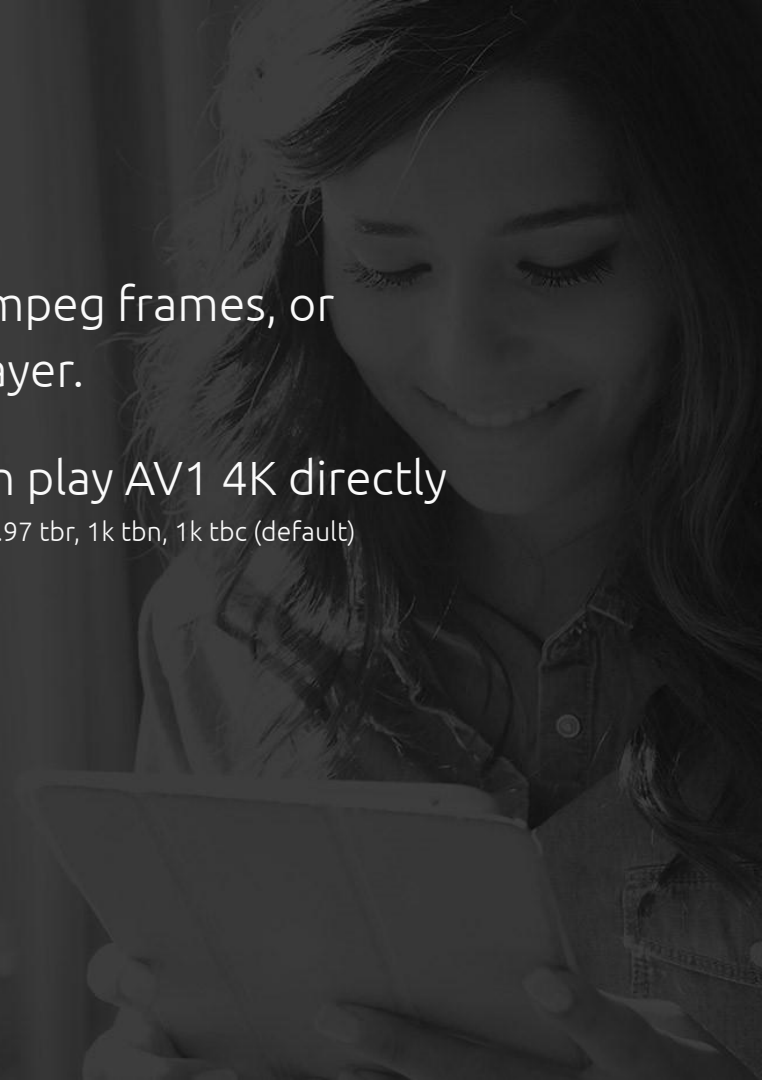
HANDLER_NAME : Core Media Video

ENCODER : Lavc58.19.102 **libaom-av1**

DURATION : 00:01:30.026000000

Mozilla Firefox Nightly (WebM)

Google Chrome Canary (WebM)

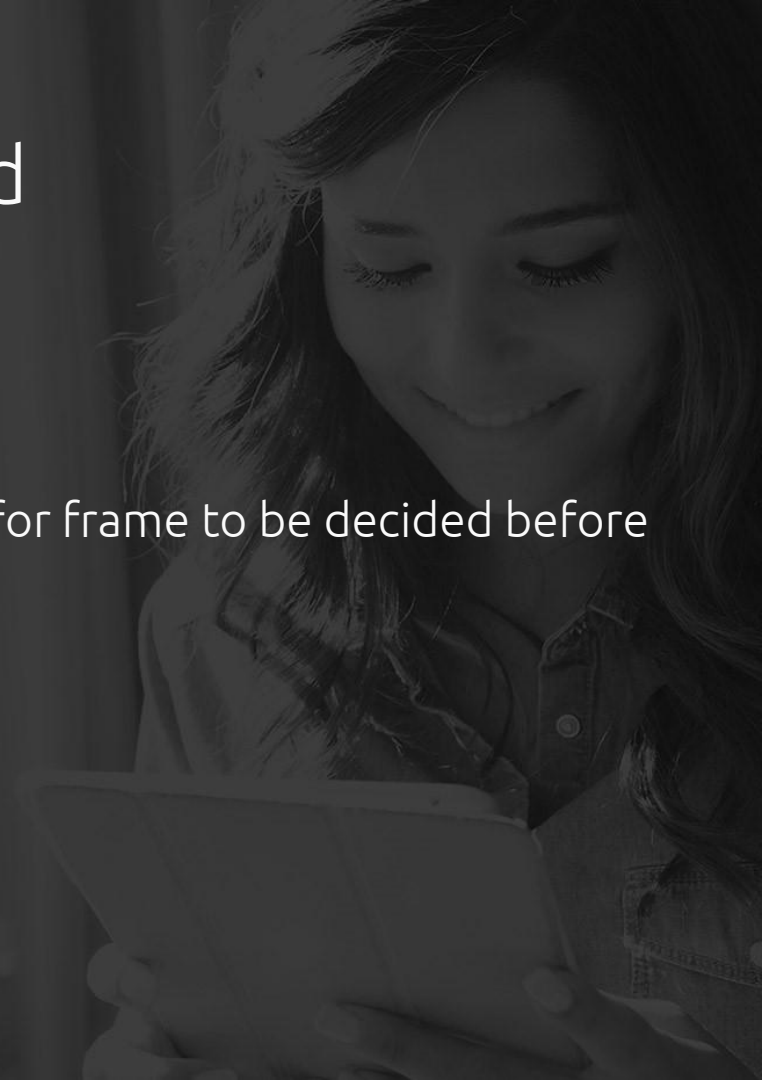


Playback: AV1 decoders: Dav1d

Dav1d

Multi pass allow multi threads to not have to wait for frame to be decided before moving to next

<https://youtu.be/UhIgBdrKyNM>



Why does AV1 *look* better? More References

Similar to MPEG, AV1 maintains concept of **Inter** and **Intra** frames

Intra frames make references to Inter frames in similar fashion, but new to AV1 is concept of the **golden frame**, which is a key frame **encoded at the highest quality**, allowing reference frames to link through to that without wasting space

As with MPEG Inter-frames also includes a compound mode - i.e. bi-prediction with other frames

The players also keep 8 references frames enough for 3 temporal or 3 spatial layers at same time off a golden frame allowing run-time dynamic choices

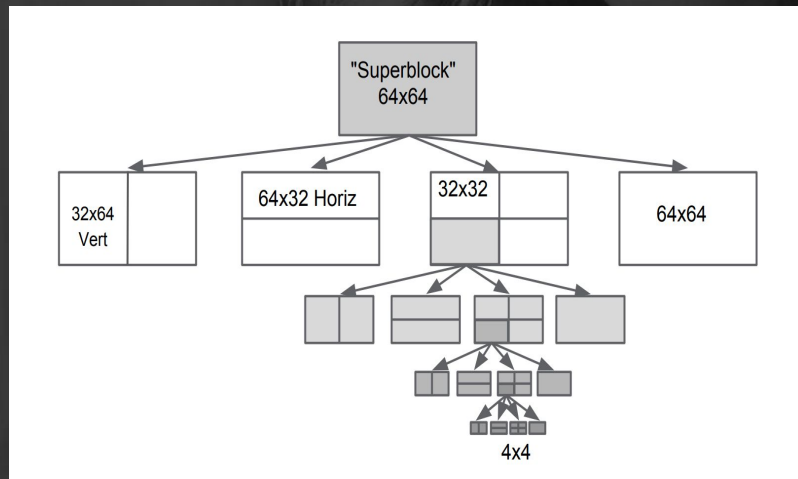
How does AV1 *compress* better? **Transforms**

Superblocks are 64x64 bits which mean 64-bit transforms. Super blocks can be dynamically split into smaller blocks in order to choose more efficient transforms which yield better results (rectangles generally compress better than squares)

Best of breed transform functions:

Discrete Cosine Transform **DCT** and
Asymmetric Discrete Sine Transform **ADST**
taken from VP9

New transforms: **flipped ADST**
and **identity transforms** are new tech
Single instruction, multiple data (SIMD)
code utilises hardware better

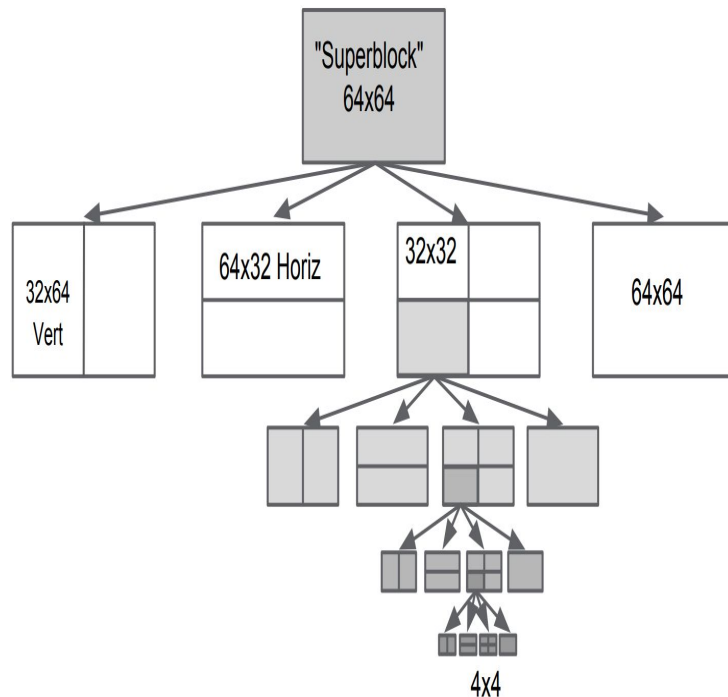


How does AV1 compress **30%** better than HEVC?

Best of breed transform functions:

Discrete Cosine Transform **DCT** and
Asymmetric Discrete Sine Transform **ADST**
taken from VP9

New transforms: **flipped ADST**
and **identity transforms** are new tech



Why does AV1 *look* better?

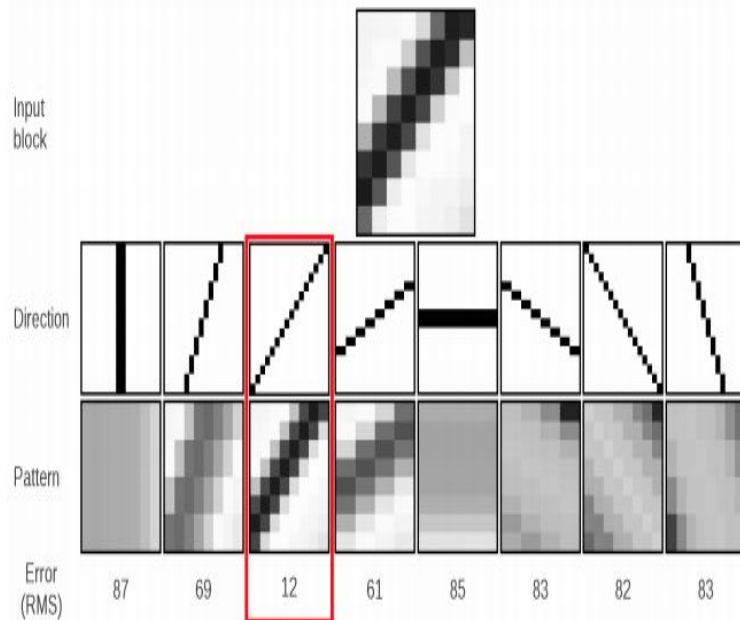
Best of breed filters combined from Daala and Thor reducing overall signal-noise-ratio
SNR

Ringing filters (from Daala)

Removes directional errors introduced by quantization and feeds back into the transforms

CLPF Constrained Low Pass Filter (from Thor)

Fixes up interpolation filter mistakes



Machine learning potential (smarter encoders)

Predictors take previous frames to determine where blocks will end up/look like

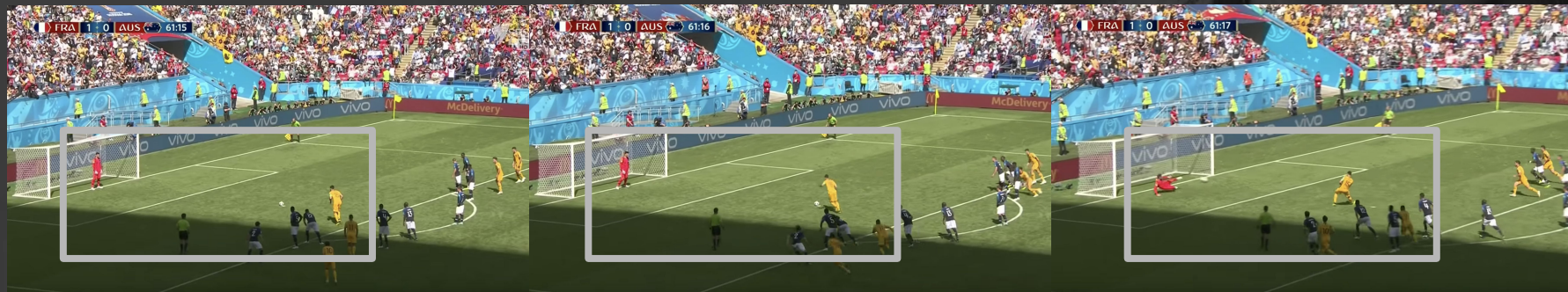
Whilst having more predictors increases encoding complexity - and so time to compress - there are more choices available which can improve overall quality of the playback.

With machine learning selecting the best predictors per asset to improve overall quality vs compression time becomes possible

Many Intra and Inter frame predictors to choose proven from codecs, e.g.

Directional, Paeth, Smooth, Palette, Blend neighbour, Edge extension, Chroma from Luma, Dynamic Reference Motion Vector Prediction (REFMV)

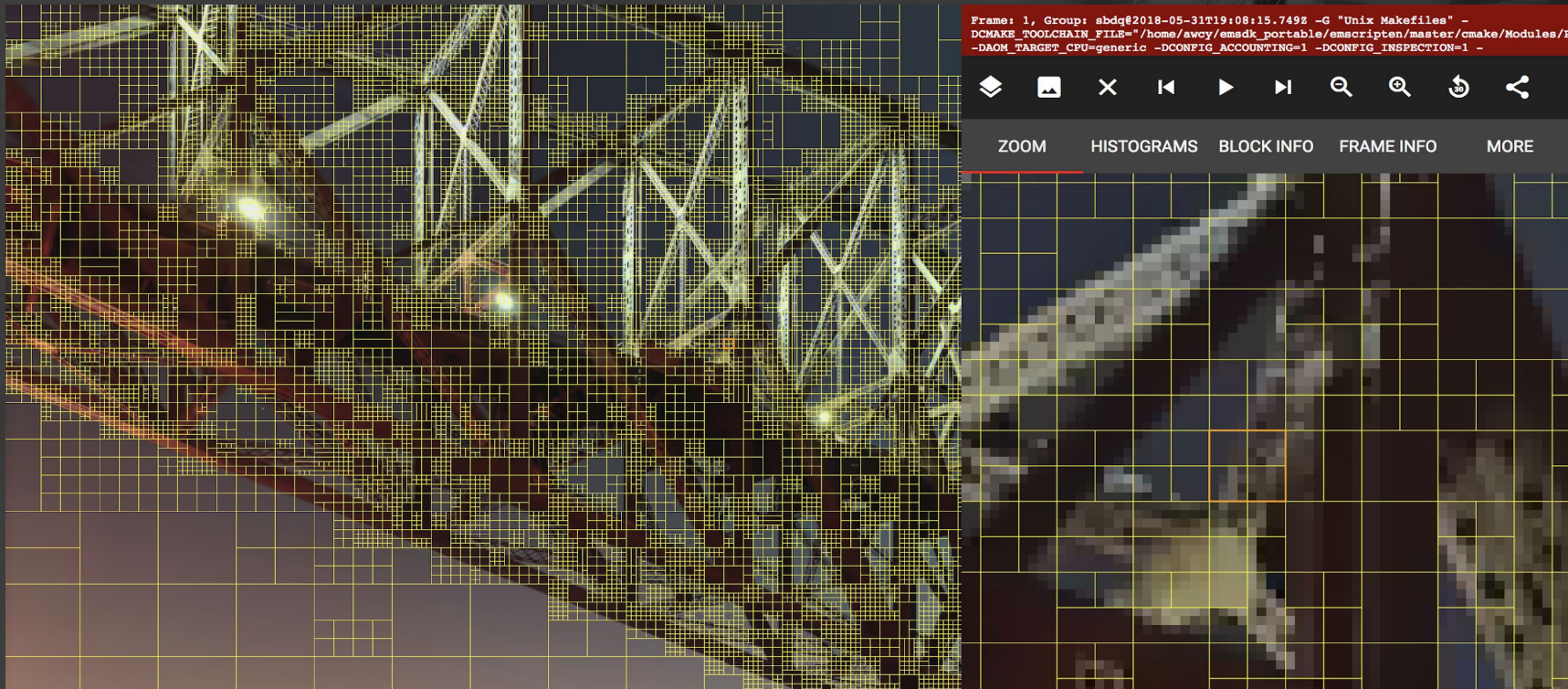
Low latency, how AV1 benefits **live sport**



AV1 tiles are groups of blocks which can be encoded, encrypted and decoded independently from other parts of the motion.

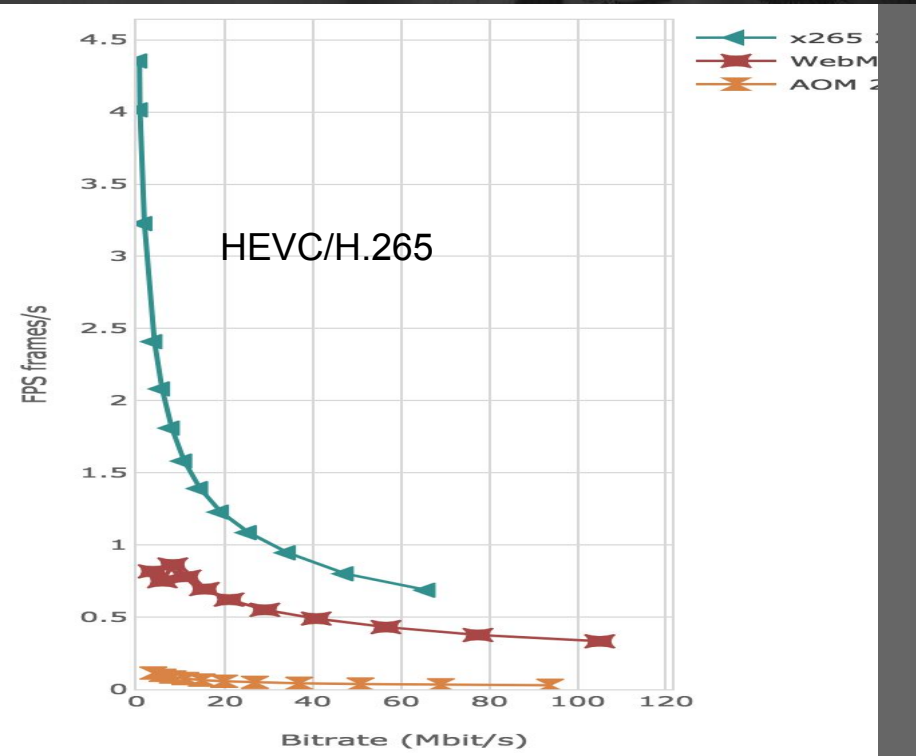
What this means is with sports in particular playback of the action can carry on even if part of the image gets lost, for example the action is more likely to continue and be watchable

How do we know/prove AV1 *is* actually better?



Where are things at today? The Elephant in the room

30% smaller! but... slow to encode



What's being done about that?

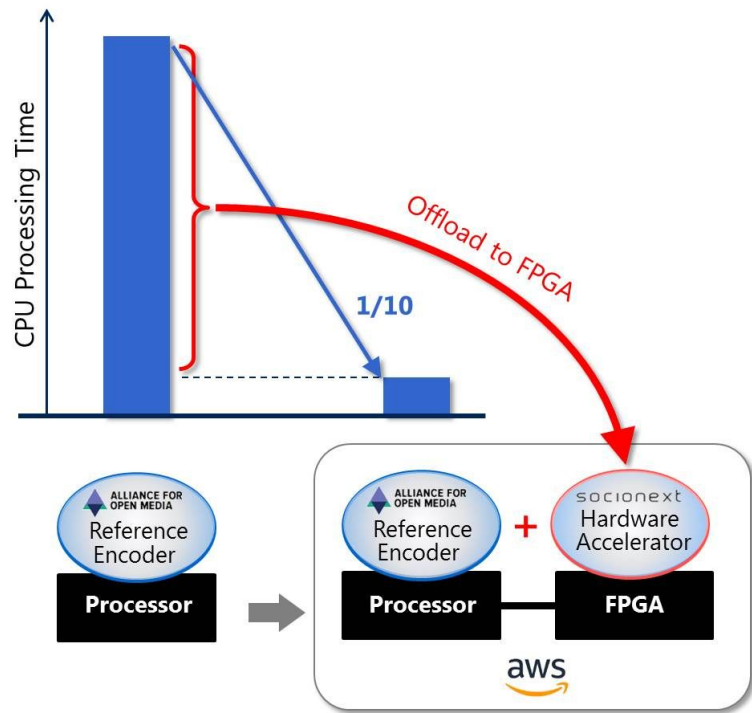
Digital Video + Audio
Data



Compressed
Video + Audio Stream



SO DIMM (204)



Some other interesting stuff, kinda

Texture segmentation methods use a deep learning based approach to detect the texture regions in a frame that is perceptually insignificant to the human visual system

High Dynamic Range (**HDR**) + Wide Colour Gamut (**WCG**) are supported to a point where colour spaces, matrices and transfer functions can be encoded directly in the bitstream itself

Less interesting... CABAC replacement (Adaptive Multi-symbol Entropy Coding), Coefficient Coding, Perceptual Vector Quantization, Golomb codes... so let's move on...

Final thought...



Alliance for
Open Media

